BANGLADESH SOFTWARE SOLUTION

BSS SWE Internship Program Plan

Target: Students / Freshers
Duration: 4 Months (16 Weeks)
Mode: 5 Days/Week (9 Hours/Day)
Outcome: Portfolio + Real Project

Program Structure

Phases	Duration	Focus
Phase 1	2 Weeks	Web Basics (HTML, CSS)
Phase 2	1 Week	Web Basics - Bootstrap
Phase 3	2 Days	Git, Github
Phase 4	2 Weeks	JavaScript Fundamentals
Phase 5	3 Weeks	Frontend Framework (React or Angular)
Phase 6	2 Weeks	Backend Basics (Node.js)
Phase 7	3 Weeks	Backend Advanced (Python FastAPI or C# .NET Intro)
Phase 8	2 days	Docker Basics
Phase 9	2 Weeks	Final Project

Program Outcomes:

- 4–6 real projects
- GitHub portfolio
- Frontend + Backend skills
- API development experience
- Team collaboration experience
- Internship completion certificate

Phase 1: Basic HTML & CSS

Tutorials / Resources

- Udemy: Modern HTML & CSS
- https://www.w3schools.com/html/default.asp
- https://www.w3schools.com/css/default.asp

Topics

HTML

- Structure, tags, forms, tables
- Semantic HTML
- Accessibility basics
- SEO basics (meta tags, Open Graph, structured data)
- Forms in depth (validation, input types, accessibility)
- Media elements (audio, video, iframe)

CSS

- CSS box model
- Inline, block, inline-block
- CSS selectors
- CSS positions (static, relative, absolute, fixed & sticky)
- Center an element (both vertically & horizontally) using flex, grid & position
- Pseudo Class & pseudo Elements
- Flex & Grid
- Transform, transition & animation
- Responsive design deeply
    - Media queries (mobile-first approach)
    - Fluid layouts (%, vw, vh, clamp)
- CSS Variables (Custom properties)
- Z-index & stacking context (very important for real UI work)
- Overflow & scrolling behavior
- Object-fit & image handling
- Advanced selectors (attribute selectors, combinators)
- Performance basics (avoiding heavy selectors, reflows)

Tasks

Task 1.1: Geometric Precision (Bangladesh Flag)

Duration: 20 Minutes

Course/Module: Modern Html CSS

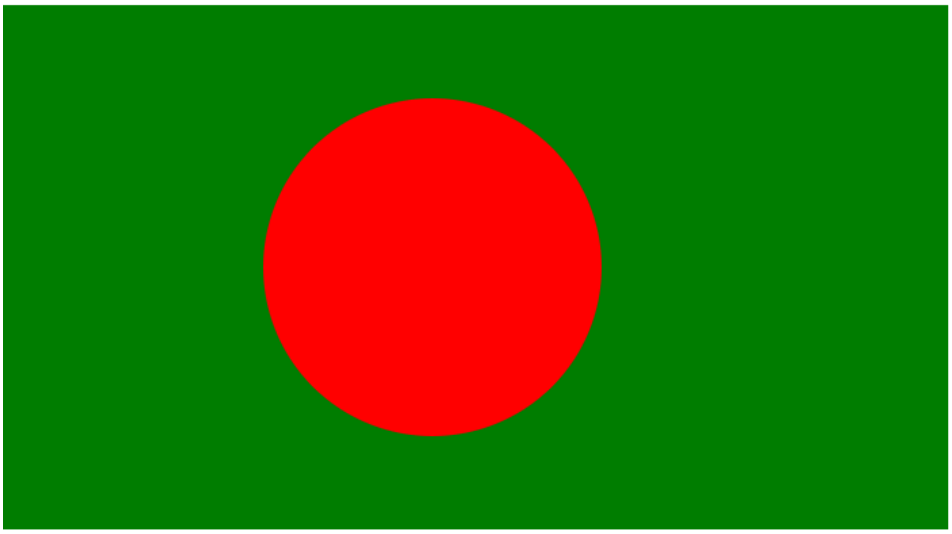
Task Description:

Construct the National Flag of Bangladesh using only CSS Position properties. This tests your ability to handle specific aspect ratios and centering.

Action Plan:

1. Create a main rectangular container. You must research and apply the official 10:6 ratio.
2. Apply the specific bottle green background color.
3. Create a perfect red circle using a separate element or a pseudo-element.
4. Position the circle so it is vertically and horizontally centered.

Demo:



Task 1.2: CSS Grid Mastery (Advanced Image Gallery)

Duration: 40 Minutes

Course/Module: Modern Html CSS

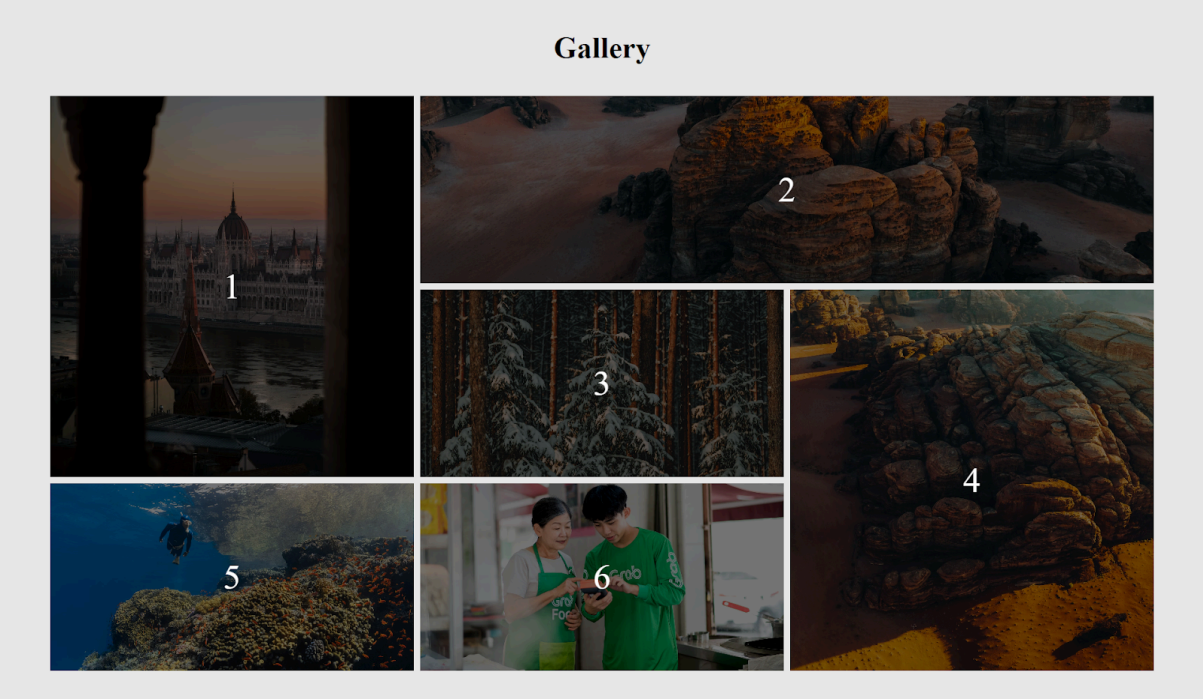
Task Description:

Build a "Bento Box" style image gallery. This task moves beyond simple grids to complex layouts where items occupy different amounts of space.

Action Plan:

1. Define a grid container.
2. Select 6 images and place them in the grid.

Demo:



Task 1.3: Structural Logic (Simple Clock UI)

Duration: 1 Hour

Course/Module: Modern Html CSS

Task Description:

Create an analog clock face. This tests your understanding of the CSS coordinate system, stacking contexts, and rotation points.

Action Plan:

1. Design a circular outer face with a distinct border.
2. Create three distinct "hands" (Hour, Minute, Second) of varying lengths and thicknesses.
3. Stack the hands in the center of the clock.
4. Adjust the "pivot point" of the hands so that when they rotate, they turn from the center of the clock rather than their own middle.

Demo:
https://drive.google.com/file/d/1_GmAtmTK-XFJPhNCyTy5PAz8wV_3A3Np/view?usp=sharing

Task 2.1: Figma Conversion - Convention Center

Duration: 2 days

Course/Module: Modern Html CSS

Task Description: Perform a full-page conversion of this landing page. Focus on standard web sections and navigation flow.

Figma Link : Convention Center Design

Action Plan:

1. **Navigation:** Build a top bar with Name on the left and menu links on the right. The navigation bar should stick to the top when scrolled.
2. **Hero:** Create a large header section with a background image and centered call-to-action text.
3. **Package Section:** Align multiple information cards horizontally using Flexbox, ensuring they stack vertically on mobile. Add hover transition to cards.

4. **Footer:** Build a clean footer with organized link columns.

Task 2.2: Figma Conversion - Restaurant System

Duration: 2 days

Course/Module: Modern Html CSS

Task Description: Convert this figma landing page design into html css landing page.

Figma Link : Restaurant Test Design

Action Plan:

1. **Typography:** Implement the specific bold/black font weights shown in the design.
2. **Hero Layout:** Create the main promotional section with high-quality imagery.
3. **Sticky Navigation:** Implement a header that stays fixed at the top, containing a search bar and utility icons.

Task 2.3: Figma Conversion - Furniture E-Commerce

Duration: 2 days

Course/Module: Modern Html CSS

Task Description: Convert Home page design into a single, polished landing page.

Figma Link: Furniture Site Design

Action Plan:

1. **Sticky Navigation:** Implement a header that stays fixed at the top, containing a search bar and utility icons.
2. **Hero Layout:** Create the main promotional section with high-quality imagery.
3. **The "Product" Section:** Use CSS Grid to display products in a 3 or 4-column layout.

4. **Feature Overlays:** Implement hover states on product cards that reveal "Add to Cart" or "Quick View" buttons without shifting the surrounding layout.
5. **Newsletter & Footer:** Build a subscription section and a multi-column footer to complete the page.

Assessment

1. Static website project (Personal Portfolio)
2. Course Related All projects
3. Figma Conversion

Phase 2: Web Basics - Bootstrap

Tutorials / Resources

- Udemy: Bootstrap 5 from Scratch
- https://getbootstrap.com/docs/5.3/getting-started/introduction/

Topics

- Grid system
- Navbar, cards, modals
- Responsive layout
- Customizing Bootstrap (overriding styles)
- Utility classes deeply (spacing, flex, display)
- Bootstrap grid breakpoints in real projects
- Forms & validation with Bootstrap
- Modal customization (Dynamic content inside modals , Modal events (open/close behavior))
- Offcanvas component - Side menus (very common in mobile UI)
- Carousel - Custom controls & autoplay settings
- Accordion - Nested accordions
- Toast & Alerts - Notifications and feedback UI
- Bootstrap Icons & Assets
    - Using Bootstrap Icons
    - Adding SVG icons
    - Icon alignment with text

Tasks

Duration: 2 days

Description: Convert Frame 4 and Frame 6 into a polished page. Must be responsive.

Figma File: Doctor house || Ecommerce

Action Plan:

1. Use Bootstrap classes only
2. If you need custom design instead of bootstrap design, customize the bootstrap classes.

Assessment

1. Course Related All projects
2. Figma Conversion

Phase 3: Git, GitHub

This phase prepares students for real-world development workflow.

Tutorials / Resources

- Git & Github
- https://www.w3schools.com/git/

Git Topics

- Git install & setup
- init, clone, add, commit, push
- branch & merge
- resolving conflicts

GitHub Topics

- Repository creation

- Pull requests
- Issues
- GitHub README & documentation

Tasks

- Push previous portfolio project to GitHub
- Team collaboration using branches

Phase 4: Basic JavaScript

Tutorials / Resources

➔ JavaScript Basics for Beginners
➔ https://www.tutorialspoint.com/javascript/index.htm
➔ https://www.w3schools.com/js/
➔ JavaScript Web Projects: 20 Projects to Build Your Portfolio

*** Interns must complete both udemy courses. The JavaScript Basics for Beginners and JavaScript Web Projects: 20 Projects to Build Your Portfolio**

Topics

- DOM Manipulation
- JS array functions (map, find, filter, reduce)
- Rest/spread operator
- Array & Object (reference type)
- Deep copy & shallow copy
- Functions (arrow function, recursive function, callback function)
- Classes & inheritance
- Encapsulation & abstraction
- Static methods
- Asynchronous JS (promise, async await, setTimeout(), setInterval())
- JSON format, JSON parse, JSON stringify
- HTTP requests and methods (get, post, put, patch, delete),
- fetch()

- Handling headers & tokens
- Error handling (status codes)
- Retry logic basics
- AbortController (cancel requests)
- Variables, variable naming rules, variable scopes
- XSS (Cross-site scripting basics)
- Input validation
- **How JS works (execution context)** **video_link**

Tasks

Duration: 3 days

Task 1: Loader

Task 2: Third-party API link for practices - JSON Placeholder

Task 3:

Full Static Website clone using HTML, CSS & JavaScript →

- Figma file or
- https://www.othoba.com

Other Recommended Resources

➔ MDN
➔ W3 schools
➔ LWS JS Playlist
➔ LWS DOM Manipulation
➔ Watch JS mock interviews on YouTube.

Phase 5: React or Angular

Option A: React

Tutorials / Resources

- Udemy
- https://react.dev/learn

- https://youtu.be/TtPXvEcE11E?si=1DirX-eC3BqBxPWn
- https://youtube.com/playlist?list=PLHiZ4m8vCp9M6HVQv7a36cp8LKzyHIePr&si=5nEqwhhvl9b3SBiY

Topics

- JSX deeply (how it compiles)
- Functional components vs class components
- Props & state management
- Event handling & forms
- Conditional rendering & lists (keys)
- Hooks (very important)
    - useState, useEffect, useRef
    - useContext (avoid prop drilling)
- Component lifecycle (with hooks)
- Lifting state up
- Controlled vs uncontrolled components
- Custom Hooks (reusable logic)
- Context API (global state)
- Performance optimization
    - React.memo
    - useMemo & useCallback
- Code splitting & lazy loading
- Error boundaries
- Portals
- Context vs Redux
- Intro to Redux
- Redux Toolkit (modern approach)
- Async state (Redux Thunk / middleware)
- Routing with React Router
- Dynamic routes
- Fetching APIs (fetch / axios)
- Tanstackquery
- Handling loading & error states

Option B: Angular

Tutorials / Resources

- Udemy

- https://angular.dev/overview
- https://youtube.com/playlist?list=PLHiZ4m8vCp9PgOOjdyNpc6AoBmKNrp_u3&si=TwdkPwBP32hkSVFT

Topics

- Angular architecture (modules, components)
- Templates & data binding (Interpolation, property, event binding)
- Directives (ngIf, ngFor, ngClass)
- Pipes (built-in & custom)
- Component communication (@Input/@Output, Services)
- Dependency Injection (very important)
- Routing (Angular Router)
- Forms (Template-driven, Reactive forms)
- RxJS (core of Angular)
    - Observables, operators (map, filter, switchMap)
- State management (NgRx basics)
- Lazy loading modules
- Guards (AuthGuard, Route protection)
- Interceptors (for HTTP requests)
- HTTPClient (GET, POST, etc.)
- Error handling & retry logic
- Environment configs

Tasks

- SPA with API
- Dashboard with CRUD UI

Assessment

✅ Frontend project (connected to mock API)

Phase 6: Node.js Basic

Tutorials / Resources

Topics

- What Node.js is (event-driven, non-blocking I/O)
- Modules (CommonJS vs ES Modules)
- File system (fs module)
- Path & OS modules
- Creating a basic server (http module)
- Package management with npm
- Environment variables (.env)
- Working with JSON & streams
- Event loop (very important)
- Buffers & streams basics
- Building APIs with Express.js
    - Routing
    - Middleware
    - Request & response handling
- RESTful API design principles
- MVC architecture basics
- Connecting to databases:
    - MongoDB with Mongoose
    - SQL basics (MySQL / PostgreSQL)
- CRUD operations
- Data validation & schema design
- Authentication (JWT)
- Password hashing (bcrypt)
- Sessions & cookies
- Basic security practices:
    - CORS
    - Rate limiting
    - Preventing XSS & CSRF
- Asynchronous programming deeply (Callbacks → Promises → async/await)
- Error handling patterns
- Custom middleware
- File uploads (Multer)
- WebSockets (real-time apps)

Tasks

- CRUD API

- Login/Register API
- Connect frontend to backend

Assessment

✅ Full MERN-style mini project

Phase 7: Python FastAPI or C# .NET Intro

Option A: Python FastAPI

Tutorials / Resources

Topics

- What FastAPI is & why it's fast (ASGI, async support)
- Creating basic APIs
- Path parameters & query parameters
- Request & response models
- Automatic docs (Swagger UI / ReDoc)
- Pydantic basics
    - Models (BaseModel)
    - Field validation
    - Default values
- Nested models
- Response models vs request models
- APIRouter (modular routes)
- Organizing large projects (folder structure)
- Tags & grouping endpoints
- Dependency Injection (Most Important)
    - Depends()
    - Reusable dependencies
    - Injecting database/session
    - Authentication dependencies

- Connecting databases:
    - SQL (PostgreSQL, MySQL)
    - NoSQL (MongoDB)
- ORM usage: SQLAlchemy
- CRUD operations
- Migrations with Alembic
- OAuth2 & JWT
- Password hashing (bcrypt)
- Role-based access control (RBAC)
- API key authentication
- CORS middleware
- async/await in FastAPI
- Async vs sync routes
- Background tasks
- Working with async DB drivers
- File uploads
- Form data
- Headers & cookies
- Custom responses (JSONResponse, StreamingResponse)
- Middleware (logging, auth, etc.)
- Startup & shutdown events
- Custom middleware creation
- Running with Uvicorn / Gunicorn
- Dockerizing FastAPI apps
- Environment variables

Option B: C# (.NET) Intro

Tutorials / Resources

- C# Basics for Beginners
- C# Intermediate: Classes, Interfaces and OOP

Topics

- C# Fundamentals (Very Important)
    - OOP (classes, inheritance, polymorphism)
    - LINQ (filtering, projection, querying)
    - Delegates & events
    - Async/await in C#

- What .NET is (runtime, CLR, cross-platform)
- Project structure in ASP.NET Core
- Creating Web APIs
- Controllers & routing
- Middleware basics
- Dependency Injection (Core Feature)
    - Built-in DI container
    - Service lifetimes:
        - Singleton
        - Scoped
        - Transient
    - Constructor injection
- Identity system
- JWT authentication
- Role-based authorization
- Claims-based authorization
- Data protection
- Request pipeline
- Custom middleware
- Built-in middleware (routing, auth, logging)
- Exception handling middleware
- appsettings.json
- Environment variables
- Dependency injection with config
- Logging (ILogger)
- Working with databases: SQL Server / PostgreSQL
- ORM: Entity Framework Core
- Code First vs Database First
- Migrations
- CRUD operations
- Filters (Action, Exception, Authorization)
- Model validation customization
- File upload & handling
- Caching (MemoryCache, Distributed cache)
- Background services (IHostedService)
- Async APIs (scalability)
- Response caching
- Compression
- Efficient database queries (LINQ optimization)

Tasks

- FastAPI CRUD service
- .NET API sample
- Compare Node vs FastAPI or Node vs .NET

Assessment

✅ Task Manager API Project

Phase 8: Docker Basics

Tutorials / Resources

- https://www.youtube.com/watch?v=b0HMimUb4f0

Topics (Basic)

- What is Docker & why
- Dockerfile basics
- Build & run container
- Docker for Node/Python app

Tasks

- Dockerize a simple Node or Python app

Assessment

- Docker container running app

Phase 9: Final Project

Interns work in teams with roles:

- Frontend Developer
- Backend Developer
- QA Tester
- Team Lead

Project

BSS Restaurant App

- BSS Restaurant API - V1
- BSS Restaurant Web - V1

- BSS Restaurant API - V2
- BSS Restaurant Web - V2

Deliverables

- GitHub repository
- Documentation
- Live demo
- Presentation